

Guia Prático: Criando um Mini-GPT no Google Colab

Aprenda a construir um modelo de linguagem do zero utilizando dados da Wikipédia e PyTorch.

Este documento traz o código completo e estruturado para criar um **Mini-GPT** baseado na arquitetura de Transformers. O modelo será treinado utilizando artigos em português extraídos diretamente da Wikipédia. Para executar o projeto, abra um novo notebook no Google Colab e altere o tipo de ambiente para **T4 GPU** (*Ambiente de execução > Alterar tipo de ambiente de execução > T4 GPU*).

Passo 1: Instalar dependências e configurar hiperparâmetros

Instalação das bibliotecas obrigatórias (wikipedia-api, tokenizers e torch) e definição das variáveis de configuração da rede neural.

```
# Célula 1: Instalação de bibliotecas e configurações
!pip install wikipedia-api tokenizers torch --quiet

import wikipediaapi
import torch
import torch.nn as nn
from torch.nn import functional as F
from tokenizers import Tokenizer
from tokenizers.models import BPE
from tokenizers.trainers import BpeTrainer
from tokenizers.pre_tokenizers import Whitespace
import os

# Configurações de hiperparâmetros
batch_size = 64      # Sequências processadas em paralelo
block_size = 128     # Comprimento máximo do contexto
max_iters = 2000     # Passos de treino
eval_interval = 200  # Intervalo para avaliação
learning_rate = 3e-4 # Taxa de aprendizado
device = 'cuda' if torch.cuda.is_available() else 'cpu'
eval_iters = 50
n_embd = 128         # Dimensão de embedding
n_head = 4           # Cabeças de atenção
n_layer = 4          # Blocos de Transformer
dropout = 0.1

print(f"Usando o dispositivo: {device}")
```

Passo 2: Coletar artigos da Wikipédia

Extração automática do conteúdo de artigos específicos da Wikipédia em português para criar a base de texto do treinamento.

```
# Célula 2: Raspagem de dados da Wikipédia
wiki = wikipediaapi.Wikipedia(
    user_agent='MiniGPT_Bot/1.0 (contato@exemplo.com)',
    language='pt'
)

# Lista de artigos para baixar
artigos = ["Inteligência_artificial", "Aprendizado_de_máquina", "Deep_learning", "Processament
o_de_linguagem_natural", "Transformadores_(modelos_de_aprendizado_de_máquina)"]
texto_completo = ""

for titulo in artigos:
    page = wiki.page(titulo)
    if page.exists():
        texto_completo += page.text + "\n\n"

# Salva localmente
with open("wiki_treino.txt", "w", encoding="utf-8") as f:
    f.write(texto_completo)

print(f"Dataset criado! Total de caracteres: {len(texto_completo)}")
```

Passo 3: Treinar o Tokenizer (BPE)

Criação de um vocabulário eficiente via Byte Pair Encoding (BPE) para agrupar caracteres comuns e transformar o texto em sequências numéricas.

```
# Célula 3: Treinamento do Tokenizer
tokenizer = Tokenizer(BPE(unk_token="[UNK]"))
trainer = BpeTrainer(special_tokens=["[UNK]", "[PAD]"], vocab_size=2000)
tokenizer.pre_tokenizer = Whitespace()

tokenizer.train(["wiki_treino.txt"], trainer)
vocab_size = tokenizer.get_vocab_size()

dados_codificados = tokenizer.encode(texto_completo).ids
dados_tensor = torch.tensor(dados_codificados, dtype=torch.long)

# Divisão Treino e Validação (90% / 10%)
n = int(0.9 * len(dados_tensor))
dados_treino = dados_tensor[:n]
dados_val = dados_tensor[n:]

print(f"Tamanho do vocabulário: {vocab_size}")
```

Passo 4: Criar o Carregador de Dados (Dataloader)

Definição do gerador de lotes aleatórios e das rotinas auxiliares para calcular o erro (loss) de treino e validação.

```
# Célula 4: Gerador de lotes (Batches)
def get_batch(split):
    data = dados_treino if split == 'train' else dados_val
    ix = torch.randint(len(data) - block_size, (batch_size,))
    x = torch.stack([data[i:i+block_size] for i in ix])
    y = torch.stack([data[i+1:i+block_size+1] for i in ix])
    x, y = x.to(device), y.to(device)
    return x, y

@torch.no_grad()
def estimate_loss():
    out = {}
    model.eval()
    for split in ['train', 'val']:
        losses = torch.zeros(eval_iters)
        for k in range(eval_iters):
            X, Y = get_batch(split)
            logits, loss = model(X, Y)
            losses[k] = loss.item()
        out[split] = losses.mean()
    model.train()
    return out
```

Passo 5: Estrutura da Arquitetura Mini-GPT

Implementação do Transformer decodificador, englobando as camadas de auto-atenção mascarada (Causal Self-Attention), camadas lineares e normalização.

```
# Célula 5: Estrutura do Modelo GPT (PyTorch)
class Head(nn.Module):
    def __init__(self, head_size):
        super().__init__()
        self.key = nn.Linear(n_embd, head_size, bias=False)
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False)
        self.register_buffer('tril', torch.tril(torch.ones(block_size, block_size)))
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        B, T, C = x.shape
        k = self.key(x)
        q = self.query(x)
        wei = q @ k.transpose(-2, -1) * k.shape[-1]**-0.5
        wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf'))
        wei = F.softmax(wei, dim=-1)
        wei = self.dropout(wei)
        v = self.value(x)
        out = wei @ v
        return out

class MultiHeadAttention(nn.Module):
    def __init__(self, num_heads, head_size):
        super().__init__()
        self.heads = nn.ModuleList([Head(head_size) for _ in range(num_heads)])
        self.proj = nn.Linear(head_size * num_heads, n_embd)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        out = torch.cat([h(x) for h in self.heads], dim=-1)
        out = self.dropout(self.proj(out))
        return out

class FeedForward(nn.Module):
    def __init__(self, n_embd):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd),
            nn.ReLU(),
            nn.Linear(4 * n_embd, n_embd),
            nn.Dropout(dropout),
        )

    def forward(self, x):
        return self.net(x)

class Block(nn.Module):
    def __init__(self, n_embd, n_head):
        super().__init__()
        head_size = n_embd // n_head
        self.sa = MultiHeadAttention(n_head, head_size)
        self.ffwd = FeedForward(n_embd)
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)
```

```

def forward(self, x):
    x = x + self.sa(self.ln1(x))
    x = x + self.ffwd(self.ln2(x))
    return x

class MiniGPT(nn.Module):
    def __init__(self):
        super().__init__()
        self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
        self.position_embedding_table = nn.Embedding(block_size, n_embd)
        self.blocks = nn.Sequential(*[Block(n_embd, n_head=n_head) for _ in range(n_layer)])
        self.ln_f = nn.LayerNorm(n_embd)
        self.lm_head = nn.Linear(n_embd, vocab_size)
        self.apply(self._init_weights)

    def _init_weights(self, module):
        if isinstance(module, nn.Linear):
            torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
            if module.bias is not None: torch.nn.init.zeros_(module.bias)
        elif isinstance(module, nn.Embedding):
            torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)

    def forward(self, idx, targets=None):
        B, T = idx.shape
        tok_emb = self.token_embedding_table(idx)
        pos_emb = self.position_embedding_table(torch.arange(T, device=device))
        x = tok_emb + pos_emb
        x = self.blocks(x)
        x = self.ln_f(x)
        logits = self.lm_head(x)

        if targets is None:
            loss = None
        else:
            B, T, C = logits.shape
            logits = logits.view(B*T, C)
            targets = targets.view(B*T)
            loss = F.cross_entropy(logits, targets)
        return logits, loss

    def generate(self, idx, max_new_tokens):
        for _ in range(max_new_tokens):
            idx_cond = idx[:, -block_size:]
            logits, loss = self(idx_cond)
            logits = logits[:, -1, :]
            probs = F.softmax(logits, dim=-1)
            idx_next = torch.multinomial(probs, num_samples=1)
            idx = torch.cat((idx, idx_next), dim=1)
        return idx

```

Passo 6: Treinar o Modelo

Execução do loop de treinamento utilizando o otimizador AdamW. A perda de validação deve ser impressa a cada ciclo configurado.

```
# Célula 6: Execução do Treinamento
model = MiniGPT().to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)

print("Iniciando treinamento...")
for iter in range(max_iters):
    if iter % eval_interval == 0 or iter == max_iters - 1:
        losses = estimate_loss()
        print(f"Passo {iter}: Loss Treino {losses['train']:.4f}, Loss Val {losses['val']:.4f}")
    )

    xb, yb = get_batch('train')
    logits, loss = model(xb, yb)
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()
```

Passo 7: Testar e Gerar Texto

Configuração de uma frase de contexto inicial (prompt) e execução da inferência autoregressiva para gerar novos tokens.

```
# Célula 7: Testando o modelo (Geração de Texto)
prompt = "A inteligência artificial é"
contexto_ids = tokenizer.encode(prompt).ids
contexto_tensor = torch.tensor([contexto_ids], dtype=torch.long, device=device)

resultado_tokens = model.generate(contexto_tensor, max_new_tokens=50).tolist()
texto_gerado = tokenizer.decode(resultado_tokens)

print("\n--- Texto Gerado pelo Mini-GPT ---")
print(texto_gerado)
```